CUSTOMER FEATURE GUIDE

Machina

Enterprise Linux hypervisor management platform.

One control plane for VMs, networks, storage, snapshots, and day-two operations on your own bare-metal KVM hosts.

by ZyvorAI Labs

© 2026 ZyvorAI Labs Private Limited · Proprietary & Confidential

WELCOME

What is Machina?

Machina is a unified control plane for virtual machines running on libvirt / QEMU / KVM. It folds scattered virsh scripting, separate console gateways, and missing fleet observability into a single Rust daemon with a web dashboard, terminal UI, REST API, and the machinactl CLI. It is built for infrastructure teams and NOC operators who run VMs on their own metal and want cloud-grade lifecycle, consoles, security, and automation without vendor hypervisor lock-in.

70+	4	12
web console screens	ways to drive it: Web, TUI, REST, CLI	workspace components, one shared core

What's inside this guide

SECTION 1

VM Lifecycle & Compute

Full create-to-delete control over every virtual machine, with live resource changes and zero-downtime moves.

Full VM lifecycle

Create, start, stop, shutdown, reboot, pause, resume, clone, rename, and delete virtual machines from any surface.

Every day-to-day VM operation in one place instead of ad-hoc virsh commands.

Dual creation backends

Build VMs either through virt-install or from native libvirt domain XML, chosen globally or per request.

Convenience for quick provisioning and full control when you need exact XML.

Golden image builder

Build reusable base images asynchronously as background jobs using virt-builder or mkosi.

Standardized, ready-to-clone images without hand-crafting each VM.

Disk & NIC hot-plug

Attach, detach, and resize disks and network cards on a running VM without a reboot.

Adjust capacity live, avoiding downtime for storage and network changes.

Live CPU & memory resize

Change vCPU count and RAM allocation on running guests, with CPU pinning and scheduler tuning.

Right-size workloads on the fly as demand shifts.

CPU & memory tuning

Fine-grained cputune, memtune, per-vCPU pinning, and scheduler policy controls.

Squeeze predictable performance out of shared hosts.

Live migration

Move running VMs between hosts with tunable max bandwidth and max downtime.

Evacuate hosts for maintenance with no service interruption.

Online block jobs

Run block commit and pull operations, and abort jobs, on live disks.

Consolidate or reshape disk chains without stopping the guest.

Guest agent by default

Optionally inject the GuestKit QEMU guest agent into every new VM for cloud-init seeding and offline changes.

In-guest coordination and reliable graceful shutdowns out of the box.

Autostart & domain XML access

Toggle per-VM autostart and read the raw libvirt domain XML for any machine.

Bring VMs back after reboots and inspect exactly what libvirt runs.

SECTION 2

Storage & Disk Management

Manage libvirt pools, volumes, and VM disks, with an optional dedicated storage control plane.

Storage pools

List, start, stop, refresh, and toggle autostart on libvirt storage pools.

Keep backing storage online and current from the dashboard.

Volume management

Create and delete storage volumes within any pool.

Provision and reclaim VM disk space without touching the CLI.

Disk images view

Browse disk images with per-VM disk resize and IO tuning.

See and reshape what each VM actually consumes.

Atlas storage integration

Connect Atlas as a storage control plane spanning Ceph, NFS, and ZFS for VM disks, snapshots, and backups.

Enterprise-grade shared storage under your fleet.

Storage tiers

Define and assign storage tiers so workloads land on the right class of disk.

Match performance and cost to each workload automatically.

Disk utility & ISO tools

Platform disk utility plus ISO creation for boot and install media.

Handle installer media and disk chores in the console.

SECTION 3

Networking

Virtual networks, host interfaces, filters, and a visual canvas for wiring the fleet together.

🌐

Virtual networks

Create, delete, start, stop, autostart, and edit the XML of libvirt virtual networks.

Full software-defined networking for guests without virsh.

🔗

Host networking

Manage host interfaces, bridges, routing tables, LLDP neighbors, and sysctl tuning.

See and shape the physical network the VMs sit on.

🚪

Port forwarding & firewall

Configure host port-forwarding rules and firewall entries.

Expose guest services safely without external tooling.

🛡️

Network filters (nwfilter)

Manage libvirt nwfilter rules that police guest traffic at the interface.

Enforce per-VM traffic policy at the hypervisor edge.

🎨

Network canvas & segments

Visually design network segments and overlays across the fleet on an interactive canvas.

Plan and understand multi-host networking at a glance.

🕸️

Network overlay & sync

Build overlay networks spanning hosts and keep network state synchronized across the fleet.

Consistent connectivity for VMs wherever they run.

SECTION 4

Consoles & Remote Access

Built-in VNC, SPICE, serial, SSH, and RDP proxies mean no separate console gateway.

🖱️

noVNC graphical console

Browser-based VNC console served directly by the daemon over WebSocket.

Reach any VM's screen from a browser, no client install.

✨

SPICE HTML5 console

SPICE remote desktop rendered in the browser via a built-in HTML5 client.

Rich desktop access with clipboard and multi-monitor support.

⌨️

Serial & terminal streams

Serial console and PTY terminal sessions streamed over authenticated WebSockets.

Debug boot issues and drop into a shell from the UI.

🔐

SSH terminal proxy

In-browser SSH terminal to VMs and ad-hoc hosts, with session TTLs.

One authenticated path to shells across the fleet.

🪟

Built-in RDP

Native RDP stream and rdp-info endpoint for Windows guests.

Windows desktops without a bolted-on gateway.

🎬

ConsoleHub session broker

Brokers console sessions with time-to-live limits, a cinema/wall multi-console view, and optional OIDC gating.

Governed, auditable console access with a NOC-style live wall.

🧲

Apache Guacamole bridge

Optional integration issuing encrypted JSON auth tokens for RDP, VNC, and SSH via Guacamole.

Plug into an existing Guacamole gateway when you have one.

📄

Virt-viewer handoff

Download a .vv connection file to open a VM in a native virt-viewer client.

Fall back to a desktop console client when preferred.

SECTION 5

Snapshots, Backup & Recovery

Point-in-time snapshots plus scheduled, retention-managed backups and disaster-recovery workflows.

Per-VM snapshots

Create, list, revert, and delete libvirt snapshots for any VM.

Instant rollback points before risky changes.

Scheduled backups

Daily backup timer exporting VMs into a backup directory or NFS target with configurable retention and optional disk inclusion.

Automated, off-box protection with no manual steps.

Backup lifecycle controls

Enable, disable, run-now, and inspect backup status and logs from machinactl.

Operate the whole backup pipeline from one command.

Fleet snapshots & schedulers

Controller-driven fleet-wide snapshot and backup scheduling across many hosts.

Consistent protection policy for the entire estate.

Disaster recovery

Dedicated DR workflows and backup targets for coordinated recovery.

A rehearsed path back when a host or site is lost.

SECTION 6

Fleet, HA & Multi-Host Control Plane

An optional enterprise controller adds HA failover, resource scheduling, and desired-state reconciliation across many hosts.

🛰

Fleet view

Lightweight peer-based fleet status, metrics, alerts, and VM inventory in the daemon.

See more than one host without the full controller.

🏢

Multi-host controller

Optional control plane persisting state (SQLite or Postgres) and driving hosts through per-host gRPC agents.

Scale from one box to a managed estate.

♻️

High availability & fencing

HA engine with health watchdog and fencing that restarts VMs elsewhere when a host fails.

Workloads survive host failures automatically.

⚖️

DRS & rebalancing

Distributed resource scheduling with placement, heatmaps, and fleet rebalancing.

Keeps load spread evenly without manual juggling.

🎯

Smart placement

Placement engine picks the best host for new or migrating VMs from live capacity.

New VMs land where they fit, not where you guessed.

🔄

Desired-state reconciliation

A reconcile engine continuously drives real state toward the declared intent.

Drift self-heals instead of piling up.

🚌

Task bus

In-memory or NATS-backed task fan-out distributing work across the fleet.

Reliable, scalable execution of fleet operations.

🔋

Fleet power & maintenance

Coordinated power control, maintenance missions, and host maintenance mode.

Drain and service hosts safely on a schedule.

🏗

Bare-metal enrollment

Enroll and validate new hosts, with bare-metal automation hooks.

Grow the fleet without hand-provisioning each node.

SECTION 7

Observability & Operations

Prometheus metrics, history, OTLP export, alerts, events, and scheduled operations keep the fleet visible and automated.

Live & historical metrics

Per-VM and host metrics with a metrics-history ring buffer persisted to disk.

Trend performance over time, not just this instant.

Prometheus scrape

A native /prometheus endpoint plus remote-write ingest for existing monitoring stacks.

Drop Machina into your Grafana dashboards immediately.

OTLP export

Export metrics, logs, and traces over OTLP/HTTP to Grafana Alloy or an OpenTelemetry Collector.

Feed one open pipeline instead of a bespoke agent.

Alerts & webhooks

Alert rules with an evaluator, notification channels, and outbound webhooks.

Get told about problems where your team already looks.

Event streaming

Server-sent event and WebSocket streams of state changes and activity.

Live dashboards and integrations react in real time.

Health & doctor checks

Deep health checks and host-readiness doctor with clear exit codes, plus a problems endpoint.

Know a deploy is healthy before users hit it.

PSI & cgroup pressure

Surfaces Linux pressure-stall and cgroup signals for hosts.

Spot resource contention before it becomes an outage.

3D topology & datacenter view

Interactive topology, datacenter, and infrastructure-graph views of hosts, VMs, and networks.

Grasp the whole estate visually in seconds.

Scheduled jobs & operations

Schedule recurring operations and VM schedules with a jobs runner.

Automate routine ops on a calendar, hands-off.

Reports & recommendations

Generated reports plus rightsizing and optimization recommendations.

Turn telemetry into concrete actions to take.

SECTION 8

Security, Compliance & SOC

A firewall control plane, network intelligence, SIEM/SOC tooling, and compliance frameworks harden the fleet.

🧱

Zeus firewall control plane

Fleet-wide firewall with profiles, drift detection, lockdown, GitOps sync, approvals, and change checkpoints.

Consistent, reviewable network policy across every host.

🐺

PacketWolf network intelligence

Kernel-native traffic discovery, ingest, and enforcement integrated as a bridge into the controller.

See and contain real traffic at the packet level.

🛰️

SOC & SIEM

Security operations center with detection, event ingest, playbooks, and a SIEM pipeline.

Detect and respond to threats without a separate SOC stack.

🎯

Threat hunting & attack surface

Threat-hunting workspace and attack-surface-management (ASM) discovery.

Proactively find exposure before attackers do.

📜

Compliance frameworks

Compliance evaluation against frameworks with remediation guidance and exportable PDF reports.

Audit-ready evidence generated from live state.

🔒

Runtime enforcement & policy

Policy engine plus runtime enforcement of security guardrails on running workloads.

Rules that actually stop bad behavior, not just flag it.

🧾

Signed audit log

Tamper-evident audit log with optional line signing, syslog, and webhook shipping, verifiable via CLI.

Prove who did what, and that the record wasn't altered.

🐧

Linux audit integration

Ingests Linux audit and SELinux AVC events with a health threshold.

Host-level security signals folded into fleet health.

🔑

Encrypted secrets & keys

Secrets management plus AES-256-GCM encryption of stored provider keys with a master key.

Credentials stay protected at rest, not in plaintext.

SECTION 9

AI & Automation (Zeus AI)

A controller-side AI engine adds natural-language ops, autonomous remediation, cost intelligence, and predictive SRE.

💬

Natural-language ops

Drive infrastructure with natural language through an intent router and NL-ops layer.

Ask for what you want instead of memorizing APIs.

👨‍✈️

Autopilot & autonomous mode

Autopilot and autonomous engines that plan and execute multi-step operations, gated by approvals.

Routine remediation runs itself, with a human veto.

🚑

Incident commander & root cause

Coordinates incident response with automated root-cause analysis and service-impact mapping.

Faster diagnosis and a clear blast radius during outages.

🔮

Predictive SRE

Predicts capacity and reliability issues and proposes SRE remediations before they bite.

Fix tomorrow's problem today.

💰

Cost & FinOps intelligence

Cost attribution, budgets, exposure FinOps, and rightsizing recommendations.

Understand and cut infrastructure spend with data.

🧭

Migration readiness

AI assessment of how ready a VM or workload is to migrate, with prechecks.

Migrate with confidence, not surprises.

🪞

Digital twin & infra memory

Maintains a digital twin and long-term infrastructure memory of the estate.

Decisions grounded in the real, remembered environment.

📚

Knowledge & runbooks

Searchable knowledge base, diagnosis, and generated runbooks for operations.

Institutional know-how available on demand.

🛒

Agent marketplace

Catalog of AI agents and pluggable LLM providers with configurable settings.

Extend automation and choose your own model backend.

🏗️

AI VM builder

Generate VM and blueprint definitions from high-level intent.

Describe a machine and let the platform draft it.

SECTION 10

Applications, Templates & Provisioning

A launchpad, marketplace, blueprints, and cloud-init studio turn raw VMs into repeatable, ready-to-run workloads.

🚀

Launchpad & spaces

App launchpad organizing workloads into spaces with per-app detail views.

Deploy and find applications like an app store, not a VM list.

🛍️

Marketplace

Built-in marketplace of deployable content and applications.

One-click access to curated, ready workloads.

🧩

Templates & catalog

Template catalog with git-backed sources, image fetch, and readiness checks.

Standardized starting points kept fresh automatically.

📐

Blueprints

Declarative blueprints describing multi-resource stacks to stamp out.

Reproduce whole environments consistently.

☁️

Cloud-init studio

Author and manage cloud-init configuration to seed guests at first boot.

Zero-touch first-boot customization for every VM.

📁

Content library

Central content store for images, ISOs, and provisioning artifacts.

One shared source of truth for boot media.

🎮

GPU command center

Discover, assign, and manage GPUs across the fleet.

Put accelerators where AI and graphics workloads need them.

🔌

USB & PCI passthrough

Pass USB and PCI devices through to guests, gated by RBAC.

Give VMs direct access to real hardware when required.

SECTION 11

Integrations & Migration

Move VMs to and from KubeVirt, OpenStack, and other clouds, and connect Machina to the wider Zyvor stack.

☸️

KubeVirt migration

Export a VM as a KubeVirt YAML bundle and apply, upload, and start it on a Kubernetes cluster.

A documented, repeatable path from libvirt to KubeVirt.

🌩️

OpenStack

Manage OpenStack instances, flavors, networks, images, and load balancers, and push local VMs to OpenStack.

Bridge on-metal VMs into your OpenStack cloud.

🔁

HyperSDK / hyper2kvm

Multi-cloud and cross-hypervisor VM migration into KVM.

Bring VMs home from other platforms.

🧪

GuestKit offline assurance

Offline VM migration assurance and guest inspection tooling.

Validate guests before and after a move.

☸️

Kubernetes & Kata

View Kubernetes workloads and Kata containers alongside VMs, with KubeVirt VNC/console proxying.

One console for both VMs and cluster workloads.

𝌆

VMware & Proxmox awareness

Controller APIs to interoperate with VMware and Proxmox sources.

Onboard estates from other hypervisors.

🧱

Zyvor platform stack

Fits with hypercluster, Zeus OS, forge, Atlas, PacketWolf, and more across the Zyvor ecosystem.

Machina is the metal layer of a full private-cloud stack.

SECTION 12

Interfaces & Administration

Four ways to operate the platform, backed by PAM/LDAP/OIDC auth, RBAC, multi-tenancy, and one-command deploys.

Web dashboard

React 19 Liquid Glass UI with classic single-host and platform multi-host shells covering 70+ screens.

A polished, complete console for the whole platform.

Terminal UI

A ratatui-based terminal client to list VMs, drive lifecycle, and watch metrics.

Full control from an SSH session, no browser needed.

REST + WebSocket API

Complete /api/v1 REST surface and /ws/v1 streams with a published OpenAPI contract at /api-docs.

Automate anything and generate typed clients.

machinactl CLI

One CLI for deps, build, install, deploy, upgrade, backup, verify, health, doctor, audit, and integrations status.

Stand up and operate a host with single commands.

PAM / LDAP / OIDC auth

Log in with Linux host accounts via PAM, with optional LDAP and OIDC SSO and Active Directory integration.

Use the identity system you already run.

RBAC roles

Admin, Operator, and ReadOnly roles mapped from a roles file, OIDC groups, or scoped API tokens.

Least-privilege access for every operator.

Scoped API tokens & sessions

Issue prefixed, scope-limited API tokens and manage HttpOnly session cookies with admin revocation.

Grant machines and people exactly the access they need.

Projects & multi-tenancy

Organize resources into projects with users, groups, and per-project scoping.

Cleanly separate teams and tenants on shared infrastructure.

Flexible deployment

Single-host machinactl, remote rsync deploy, multi-host controller/agent, or a daemon-only Helm chart.

Install the way that fits your environment.

Support & upgrade tooling

In-console support bundle, upgrade flows, and self-updating deploys with health verification.

Maintain and troubleshoot the platform from inside it.

GETTING STARTED

Your first 30 minutes

A suggested path to value on day one.

1. **Prepare a Linux KVM host**
   Run ./machinactl doctor to confirm /dev/kvm, systemd, libvirtd, and required tools are present. Build only on Linux.
2. **Deploy in one command**
   Run ./machinactl deploy to install dependencies, build a release, install, start the daemon, and smoke-test the API.
3. **Open the console**
   Browse to https://<host>:5092 (self-signed TLS by default) and sign in with a Linux host account via PAM, or launch the TUI with machina.
4. **Lock down access**
   Populate /var/lib/machina/roles.json (an empty file makes everyone admin), set a strong MACHINA_JWT_SECRET, and choose PAM, LDAP, or OIDC.
5. **Turn on protection & telemetry**
   Enable scheduled backups with ./machinactl backup enable, point them off-box, and wire Prometheus scrape or OTLP export.
6. **Scale to a fleet (optional)**
   Install the controller and agents with INSTALL_PLATFORM=1 or deploy-remote --platform to unlock HA, DRS, and the platform UI.

Support & contact

Machina is developed by **ZyvorAI Labs**. For onboarding, a proof-of-concept, or a guided demo, contact your account team at **info@zyvor.dev**.

> **Good to know:** Machina builds and runs on Linux only (it depends on libvirt/QEMU/KVM headers and /dev/kvm) — the workspace does not compile on macOS, though the web UI alone does. HTTPS ships with a self-signed certificate that should be replaced with a CA-signed one for production. Security defaults matter: an empty roles.json grants every user admin, and the dev-only auth-bypass flags must never be set in production. The multi-host controller and agent tier is systemd-only (the Helm chart deploys just the daemon), and SAML is config-only today. Integrations such as OpenStack, KubeVirt, Guacamole, PacketWolf, and Atlas are disabled by default and require their own endpoints or credentials.

Machina

One control plane for VMs, networks, storage, snapshots, and day-two operations on your own bare-metal KVM hosts.

info@zyvor.dev · zyvor.dev